

1

2

# NOCKONOMICON

3

N. E. Davis

4



## 5 Contents

|    |          |                                      |           |
|----|----------|--------------------------------------|-----------|
| 6  | <b>1</b> | <b>The Excavation</b>                | <b>3</b>  |
| 7  | 1.1      | Tabulation . . . . .                 | 5         |
| 8  | 1.2      | Solid-state indices . . . . .        | 6         |
| 9  | 1.3      | Alchemy and Process . . . . .        | 7         |
| 10 | 1.4      | Generalization . . . . .             | 9         |
| 11 | 1.5      | The Universal Machine . . . . .      | 10        |
| 12 | 1.6      | Pure Statements . . . . .            | 11        |
| 13 | 1.7      | The Ruliad . . . . .                 | 12        |
| 14 | <b>2</b> | <b>The Letters</b>                   | <b>15</b> |
| 15 | 2.1      | Combinators in Computing . . . . .   | 16        |
| 16 | 2.1.1    | The SKI calculus . . . . .           | 18        |
| 17 | 2.1.2    | The BCK $\omega$ calculus . . . . .  | 22        |
| 18 | 2.1.3    | $\Upsilon$ Fixed-Point . . . . .     | 23        |
| 19 | 2.1.4    | $\imath$ Universal . . . . .         | 26        |
| 20 | 2.2      | Nock Opcodes as Combinator . . . . . | 29        |

---

|    |          |                                        |           |
|----|----------|----------------------------------------|-----------|
| 21 | 2.2.1    | I Identity = Opcode 0 . . . . .        | 33        |
| 22 | 2.2.2    | K Constant = Opcode 1 . . . . .        | 34        |
| 23 | 2.2.3    | S Substitution = Opcode 2 . . . . .    | 35        |
| 24 | 2.2.4    | The Axiomatic Operators . . . . .      | 35        |
| 25 | 2.2.5    | Pythagoras and Peirce in Nock . . . .  | 39        |
| 26 | 2.3      | Idioms & Design Patterns . . . . .     | 42        |
| 27 | 2.3.1    | C Flip Operator . . . . .              | 42        |
| 28 | 2.3.2    | B Composition Operator . . . . .       | 44        |
| 29 | 2.3.3    | ⌈ Join Operator . . . . .              | 44        |
| 30 | 2.3.4    | U Duplication Operator . . . . .       | 45        |
| 31 | 2.4      | Nock as Generally Computable . . . . . | 46        |
| 32 | 2.5      | A Stateful Nock Kernel . . . . .       | 47        |
| 33 | <b>3</b> | <b>The Virtues</b>                     | <b>51</b> |
| 34 | <b>4</b> | <b>Interpreter Runtime</b>             | <b>55</b> |
| 35 | <b>5</b> | <b>Moon: A Higher-Level Language</b>   | <b>57</b> |
| 36 | 5.1      | Relationship of Moon to Nock . . . . . | 57        |
| 37 | 5.2      | ☾ New Moon . . . . .                   | 59        |
| 38 | 5.2.1    | Design Criteria . . . . .              | 59        |
| 39 | 5.2.2    | New Moon Examples . . . . .            | 60        |
| 40 | 5.2.3    | Parsing New Moon . . . . .             | 61        |
| 41 | 5.2.4    | Compiling New Moon . . . . .           | 62        |
| 42 | 5.3      | ☾ Crescent Moon . . . . .              | 62        |
| 43 | 5.4      | ☾ Full Moon . . . . .                  | 63        |

---

|    |                                          |           |
|----|------------------------------------------|-----------|
| 44 | <b>6 A Virtual Machine Runtime</b>       | <b>65</b> |
| 45 | 6.1 The Event Loop . . . . .             | 66        |
| 46 | 6.1.1 A Simple State Machine . . . . .   | 66        |
| 47 | 6.1.2 An Evolved State Machine . . . . . | 68        |



---

## 48 Foreword

49 There are many worthy raconteurs of the history of comput-  
50 ing, such as Hofstadter (1980) and Penrose (2004). What distin-  
51 guishes my approach to the history of computing from other  
52 tellings? Computing has deep connexions to alchemy and true  
53 language, which I hope to demonstrate without assuming any  
54 Hermetic background on the reader's part. Much like tech-  
55 nology, computing is not merely a tool but a stance towards  
56 the world. Moreover, it is innate (Wolfram's principle of com-  
57 putational equivalence). However, it does not entail technol-  
58 ogism's supercession; computing is always both a lens and  
59 a kaleidoscope, through which many layers of reality can be  
60 seen simultaneously. It is a way of seeing the world as a sys-  
61 tem of pure statements and acting to make those statements  
62 true. It is both ontology and epistemology.

63 First, therefore, I must establish that computation is the  
64 apex of a long human search for a language of truth. The ori-  
65 gin of symbol itself in the σύμβολον, a token broken in half to

## NOCNONOMICON

---

66 infallibly mark identity, is one font. Later “symbol” was the  
67 profession of faith, both an attempt to formulate truth and a  
68 sign of shared claims. The primary source, however, is lan-  
69 guage itself, Hofstadter’s “strange loop” of self-reference. The  
70 first possible Nock formula is a crash, the void. The second is  
71 self-reference, awakening. Peirce would be proud of the pro-  
72 gression. We have tricked ourselves into thinking that com-  
73 puting is a late development, a fruitful elaboration of the end  
74 of mathematics. In fact, it is the philosopher’s stone, and it  
75 underlies every process in the world. It has turned lead into  
76 gold.

77     The historical account which follows aims throughout at  
78 the goal of understanding what Nock is and why it matters as  
79 a protocol. The reader will understand what mankind has been  
80 looking for, when we have caught various pieces of the puzzle,  
81 how these things came together in the mid-20th century, and  
82 how they have been refined into a pure machine.



---

## Chapter 1 The Excavation

Machines are pure products of human intellect, constructed for man's own purposes. They are nowhere to be found in the world of nature (as products of nature); yet the workings of the laws of nature find their purest expression in machines, purer than in any of the products of nature itself. The laws of nature work directly in machines, with an immediacy not to be found in the products of nature. (Keiji Nishitani, *Religion and Nothingness*, 1982, pp. 129–130)

The sky above the port was the color of television, tuned to a dead channel. (William Gibson, *Neuromancer*, 1984, p. 1)

The clay of Mesopotamia is dug out (frequently from a

98 riverbank or canal), mixed with water, and left to sit.<sup>1</sup> The  
 99 coarse pebbles settle to the bottom, while organic impurities  
 100 float to the top. The fine clay in the middle is kneaded and  
 101 shaped into loaves from which handful were broken off and  
 102 flattened into tablets. The best clay has a balance of loess for  
 103 structure and silicate for surface smoothness.

104 Into this clay, the scribe would press a stylus to make marks.  
 105 Most styli were made of reed, with a triangular tip that pro-  
 106 duced the characteristic wedge-shaped marks of cuneiform writ-  
 107 ing; some, however, were of bone or brass. The pattern of  
 108 marks represented words, syllables, or numbers, and the scribe  
 109 would use a combination of these to record information. Im-  
 110 portant tablets would then be dried in the sun or baked in a  
 111 kiln to harden them, making them last for millennia.

112 While it's not obvious to us today with paper and screens,  
 113 clay was in many respects a logical medium for recording and  
 114 processing information. It was malleable, reusable, and could  
 115 be dried to preserve its state indefinitely. It was also abundant  
 116 and accessible, making it an ideal material. As a *medium*, it  
 117 was the “thing between” both the scribe and the information,  
 118 and between the scribe and the reader. It was a medium for  
 119 communication, but also a medium for thought itself.

120 The first human act of magic was the spoken word, the  
 121 conveyance of thought from one mind to another immediately.

---

<sup>1</sup>[ (I)pp. 297–298]Taylor2011.

The second was the written word, the conveyance of thought from one mind to another at a distance in space and time. The clay tablet was in a real sense the first computer.

### 1.1 Tabulation

Most of the earliest cuneiform tablets are not records of transactions or stories, but rather tables of numbers. The first widespread use of writing was to track quantities. The scribe would make a mark on the clay to represent a quantity, and then group those marks together to represent larger quantities. We think of this as mundane because Pythagoras won so thoroughly that numbers are invisible to us. But the scribe was not just recording a number, but rather creating a new entity: a number that could be manipulated and transformed according to rules. The scribe was not just writing, but rather inventing the first programming language.<sup>2</sup>

The spreadsheet appears early in human history: the Babylonians used tables on cuneiform tablets to keep track of their agricultural production, their trade, and their taxes. The cuneiform tablet is not merely a record, but a state. The clay remembers what the scribe knew, and yields it back unchanged to any reader who applies the correct formula. This is no metaphor: it is the oldest known implementation of a key-value store, fired

---

<sup>2</sup>Neal Stephenson told no lies in *Snow Crash*.

144 in a kiln and buried for four thousand years, waiting.

145       Likewise, the earliest descriptions of procedures date back  
146 to the same era of antiquity. Particular problems were solved  
147 by following a set of instructions: if you needed the area of a  
148 field or the volume of a silo, you followed a recipe. It was not  
149 until the Egyptian Rhind Mathematical Papyrus, however, that  
150 the idea is generalized to a procedure that could be applied to  
151 any input. This is the first general algorithm, the first function,  
152 the first recorded computer program. Humans using papyrus  
153 and tablets were still the “hardware” of the machine, but they  
154 were following a program that was independent of their par-  
155 ticular circumstances. The procedure was a pure statement of  
156 transformation, and the scribe was an interpreter of that state-  
157 ment. The written record is RAM, ROM, and a hard drive all at  
158 once. It is the first database, the first spreadsheet, and the first  
159 algorithm. It is the first computer.

## 160 **1.2 Solid-state indices**

161 The downside of clay is that it is slow to write and, although  
162 it can be recycled, it is not reusable in the moment. Papyrus  
163 was even less reusable and expensive to manufacture despite  
164 its speed advantages. This situation led to the development of  
165 temporary and manipulable physical representations, such as  
166 tokens and abaci. The abacus stored the intermediate steps of  
167 a calculation in physical space as the location of beads on a

rod, with the “meaning” of the beads determined not only by their position and the rules of manipulation, but also by the intent of the user. The abacus was a physical manifestation of the spreadsheet, a way to keep track of state and perform operations on it without having to write anything down. It was a register machine, with the beads as registers and the rods as the operations that could be performed on them. An expression or equation was not merely notional, but embodied in the physical arrangement of the beads. The abacus was a physical computer, and the scribe was its operator.

### 1.3 Alchemy and Process

This early success in moving a calculation from the mind to a physical representation spurred more ambitious attempts to mechanize the world—whether the rules of manipulation could themselves be encoded in the device. The mad Franciscan monk Ramon Llull designed a machine of rotating concentric wheels inscribed with symbols that could be combined to generate true statements about God and the universe. Gottfried Leibniz built a mechanical calculator, the Stepped Reckoner, and dreamed of a universal language in which all true statements could be proven by calculation. (Leibniz: “Some selected men could finish the matter in five years”<sup>3</sup>; he also introduced the

---

<sup>3</sup>[ (I)p. 224]Loemker1969.

190 use of I Ching binary numerals.) These machines were the first  
 191 compilers, the first attempts to mechanize truth itself into rep-  
 192 resentation. (They were also the first attempts to mechanize  
 193 creativity, to build a machine that could generate new insights  
 194 and ideas rather than simply execute pre-defined instructions.)

195 Nor were they the first or the last: the alchemists of the  
 196 Middle Ages sought to transmute base metals into gold, and in  
 197 doing so they developed a rich set of processes and transforma-  
 198 tions that could be applied to substances. Alchemy described  
 199 a set of transformations that could be applied to substances to  
 200 achieve certain effects. These processes (*albedo*, *rubedo*, *citrini-*  
 201 *tas*, and *nigredo*) were underspecified, from our modern chem-  
 202 ical viewpoint, and relied too much on external appearances.  
 203 What, for instance, is the scope of *rubedo*? *Albedo* is rela-  
 204 tive rather than an absolute descriptor. But what these pro-  
 205 cesses pointed towards was fundamentally correct: processes  
 206 can be described in sets of primitive operations, and these op-  
 207 erations can be implemented in a recipe or program. They  
 208 had the wrong set of primitives and operators, but the right  
 209 idea. Alchemy has been described as a proto-science, but it  
 210 was also proto-programming: it was an ill-formed, inchoate  
 211 pseudocode for reality that required refinement over its prim-  
 212 itives, its operators, and its goals in order to become a true  
 213 language of transformation.

## 1.4 Generalization

Llull and Leibniz had built sophisticated toys; like Hero of Alexandria's aeolipile as ancient jet engine, their potential had not penetrated a broader consciousness. French weaver Joseph Marie Jacquard saw in the repetitive mechanical procedures of the shuttle and loom a new possibility: that of encoding a fabric pattern as a collection of thread "instructions" using a punched cardboard card—a transformation worthy of the alchemists. Jacquard's loom was the first true compiler, a machine that could take a set of instructions and produce a complex output via transformation into another medium. Almost incidentally, it was also the first machine to use punched cards, which would later be used in early computers for input and storage. Jacquard's loom was a physical manifestation of the idea of generalization: it took a specific pattern and abstracted it into a set of instructions that could be applied to any pattern. It was the next concrete step towards the universal machine.

English inventor Charles Babbage saw in the loom a prototype for generalizing all kinds of processes. He designed the Difference Engine, a mechanical calculator that could compute polynomial functions, and later the Analytical Engine, a more general-purpose machine that could be programmed using punched cards. Babbage's machines were the first truly mechanized general-purpose computers, in the sense that they were designed to be programmable and to execute a wide range

239 of instructions. The Analytical Engine's mill was a central pro-  
 240 cessing unit, its store was memory, and its punched cards were  
 241 both input and storage, distant heirs of the abacus and the  
 242 Mesopotamian clay.

## 243 1.5 The Universal Machine

244 In the twentieth century, the logicians and mathematicians  
 245 and engineers set to work to unite the various strands of Leib-  
 246 niz's dream and Babbage's machines into a single, coherent vi-  
 247 sion of a universal machine. At first they were less concerned  
 248 with the physical device (although this soon became signifi-  
 249 cant again); they sought a concise formal description of the  
 250 universal machine itself, a set of primitives and operators that  
 251 could be used to express any computable function. Alan Tur-  
 252 ing's Turing machine, Kurt Gödel's recursive functions, Kon-  
 253 rad Zuse's Plankalkül, and John von Neumann's architecture  
 254 were all different approaches to this problem, but they con-  
 255 verged on the same conclusion: there is a simple, abstract ma-  
 256 chine that can compute anything that is computable. This was  
 257 the birth of the modern concept of computation, and it laid the  
 258 groundwork for the development of actual physical computers.

259 At the heart of this transition from dream to waking, from  
 260 the alchemical to the computational, Turing and Gödel struck  
 261 two fatal blows to the previous conception. Turing showed  
 262 that there is no general algorithm for determining whether a



given program will halt or run forever, and Gödel showed that there are true statements in mathematics that cannot be proven within any given formal system.<sup>4</sup> These results shattered the dream of a complete and consistent formal system that could capture all mathematical truths, and they also showed that there are limits to what can be computed. However, they also clarified the nature of computation and the universal machine, and they set the stage for the development of actual computers. The dream of a universal machine found its own limits and in doing so defined itself exactly.

### 1.6 Pure Statements

At the same time as the formal process itself was undergoing definition, Moses Schönfinkel, Haskell Curry, Alonzo Church, and Stephen Kleene were working on the problem of finding a minimal set of rules (called combinators) that could be used to express any computable operation. These combinators were pure statements of transformation which could be combined in various ways to produce complex computations. Unlike alchemical processes, these were well-defined and unambiguous; they likewise bore little resemblance to physical processes. They were not designed to be implemented in any particular

---

<sup>4</sup>Indeed, Gödel spent many years believing that Leibniz's work on the *characteristica universalis* had been suppressed by his biographers.

284 machine, but rather to capture the essence of computation it-  
 285 self. They became the first pure programming language, in the  
 286 sense that they were a formal system for expressing compu-  
 287 tations without reference to any particular machine or imple-  
 288 mentation. They were pages from *The Book*.

289 And so, before there were modern computers there were  
 290 combinators, and before there were combinators there was the  
 291 question: what is the minimum number of operators required  
 292 to compute anything computable? It turns out that you can  
 293 in fact nail it down to one (the  $\iota$  combinator) at the cost of  
 294 extreme obfuscation, but a more practical answer is two: S and  
 295 K. Two runes, all else is commentary. (What does it mean that  
 296 there are one, or two, rather than ten or fifty? An ontology  
 297 and theology of the combinators lurks within.) Virtually all  
 298 systems elaborate more, sometimes many more, operators and  
 299 primitives to make computation legible and tractable.

## 300 1.7 The Ruliad

301 Computer scientist Stephen Wolfram has argued that the uni-  
 302 verse itself is a kind of computation, and that the laws of physics  
 303 are the rules of this computation. He has experimented with  
 304 various simple computational systems, such as cellular automata,  
 305 and found that they can produce complex behavior that re-  
 306 sembles the complexity of the natural world. This “new kind  
 307 of science” has suggested the Ruliad, or a hypothetical limit

of all possible computations, as a way to understand the fundamental nature of reality.<sup>5</sup> The Ruliad is a kind of computational multiverse, in which every possible computation is represented as a point in a vast space of possibilities: “the result of following all possible computational rules in all possible ways”.<sup>6</sup>

We have now excavated the roots of computing and found that they were always there. The universal machine was always implicit in language, in mathematics, and in the machine. Mankind’s intellectual history is merely the vector by which it excavates itself. Any non-trivially complex physical process is already performing universal computation—the principle of computational equivalence; therefore, the universe has always been computing, in every grain of sand and collision of particles. What we call “computing” is not a human invention but a *recognition*: we have learned to read a text that predates us by 13.8 billion years. Computing kept trying to instantiate itself in purer and purer forms, as we cleaned its gears and polished its brass—and it succeeded. We have learned to speak the language of the universe, and in so doing we have found that the roots of computing run into the pages of the cosmos itself.

Nock is one extremely pure, minimal, frozen dialect of that universal text, but it is not the only one, as we have seen; its

---

<sup>5</sup>Wolfram (2002); Wolfram (2021).

<sup>6</sup>Wolfram (2021).

## NOCKONOMICON

---

331 concision and expressiveness serves as our point of embarking  
332 into the Ruliad. The version of the Nock specification we use is  
333 called Nock 4K—meaning that it has four revisions remaining  
334 before it is sealed, a countdown frozen into its name. Nock 4K  
335 is one particular field of points in the Ruliad, a simple and ele-  
336 gant language that captures the essence of computation itself.  
337 Four revisions remain, then it freezes forever. Let us begin.

---

## Chapter 2 The Letters

Prometheus is creative in the modern sense: his thought has a forward momentum that propels it beyond the bounds of divine decree. With an eye toward his own piece of the divine action, he schemes, plans, and analyzes, and thus abandons the more contemplative “afterthought” mode of being whereby insights are caught on the cosmic rebound. In a large sense, Prometheus learns to think for himself—learns to kindle his own intellectual fire rather than bask in the fire-illuminated expanse of the gods. (David Grandy, *The Speed of Light: Constancy and Cosmos*, 2009, p. 159)

Those who search for gold dig over much earth and find a little. (Heraclitus the Obscure, DK 22B22, tr. Jonathan Barnes)

We could begin our practical exploration of computing with

355 cuneiform, but we will elide several millennia and skip ahead  
 356 to the combinator, the most recently excavated component of  
 357 computing. The combinator is a mathematical concept, a func-  
 358 tion that takes one or more arguments and returns a result  
 359 without referring to any variables outside its own arguments.  
 360 A combinator system demarcates a self-contained field of play  
 361 within the Ruliad. This fundamental approach lets us build a  
 362 computer out of combinators alone, which we can then elabo-  
 363 rate with real hardware into a practical machine.

364     The first section of this chapter will introduce the basic SKI  
 365 and BCKW combinator calculi and examine how they can mutu-  
 366 ally represent each other. The second section will introduce  
 367 Nock’s opcodes as combinators, and show how they can be  
 368 used to express various combinator idioms. The third section  
 369 will show how to build practical affordances for common pro-  
 370 gramming tasks, such as conditionals, recursion, subprocedure  
 371 invocation, and data structures. The final section will discuss  
 372 the relationship between combinators and virtualization, and  
 373 how Nock can be used to build a virtual machine even in a  
 374 crash-only context.

## 375 2.1 Combinators in Computing

376 The pioneers of computing elaborated (at least) three comple-  
 377 mentary visions:

1. The Turing machine, a simple abstract machine that can compute anything computable.
2. The lambda calculus, a pure functional language based on substitution and abstraction.
3. Combinatory logic, a variable-free functional language based on combinators.

Logicians soon demonstrated the equivalence of these approaches, but particular formulations are still preferred for particular tasks. (And because these are formally equivalent, many theorems can be proven about the easiest case to reason with, then the conclusion transposed to the others.) The Turing machine is the most intuitive model of computation, and is often used to prove computability results. The lambda calculus is the basis of many functional programming languages, such as Lisp, Scheme, and Haskell. Combinatory logic is less widely used in practice, but has a long history in mathematical logic and type theory. It is also the basis of Nock.

Before we delve deeper into the “Turing tarpit”,<sup>1</sup> let’s talk about what a combinator is. At its simplest, a combinator is a function that takes one or more arguments and returns a result, without referring to any variables outside its own arguments. In practice, we’ll use different combinators to build

---

<sup>1</sup>Perlis (1982).

up more complex functions, and we'll see informally how they can be combined to express any computable function. We can even build combinators out of other combinators. The objective of combinatory logic is to express all computation in terms of combinators alone, without the need for variables or named functions.

This first section will introduce some elemental and compound combinators and make the case that they circumscribe “computing” as a concept. There are several systems which can be built out of combinators, but we will focus on the SKI and BCK systems, which are the most well-known and widely studied. We will also introduce Chris Barker’s universal  $\iota$  combinator, which yields all the others. Finally, we will show how Nock’s opcodes can be used as combinators to express various combinator idioms.

## 2.1.1 The SKI calculus

Computable functions can be expressed using minimal sets of combinators. Many logicians today prefer to work with Moses Schönfinkel and Haskell Curry’s SKI combinator calculus, which has three combinators. (We previously introduced this as the SK system with two combinators; the I combinator is often included as a primitive to ease calculations, but it can be derived from S and K alone, so we will treat it as a primitive combinator for now.)



The identity combinator  $I$  is conventionally written  $Ix = x$ . This simply means that it returns its single argument unchanged. There is little to say of this combinator at this point, but logicians have found that it frequently simplifies expressions.

---

$I\ x\ ==\ x$

---

The constant combinator  $K$  is conventionally written  $Kxy = x$ . This means that it takes two arguments and returns the first, ignoring the second. It is useful for building functions that always return a fixed value.

### Questions

1. What is the result of  $I\ x$ ?
2. What is the result of  $I\ I\ x$ ?

---

$K\ x\ y\ ==\ x$

---

The substitution combinator  $S$  is conventionally written  $Sxyz = xz(yz)$ . This means that it takes three arguments: a function  $x$ , and two values  $y$  and  $z$ . It applies the function  $x$  to the value  $z$ , and also applies the function  $y$  to the value  $z$ , then applies the result of  $x\ z$  to the result of  $y\ z$ . This combinator is more complex than the others, but it is very powerful, as it allows us to express function application and composition.

449  
450  $S\ x\ y\ z == x\ z\ (y\ z)$

---

452       The SKI system is Turing complete ( $= \mu$ -recursive), mean-  
453 ing that any computable function can be expressed using only  
454 these three combinators. It is also equivalent to the lambda cal-  
455 culus, meaning that any lambda expression can be translated  
456 into an equivalent SKI expression and vice versa.<sup>2</sup>

457       Combinators can be combined to form more complex ex-  
458 pressions. For example, the combinator  $S\ K\ K$  is equivalent to  
459 the identity combinator  $I$ , which can be proven thus:

460  $S\ K\ K\ z == \quad ;\ x \rightarrow K,\ y \rightarrow K,\ z \rightarrow z$   
461  $K\ z\ (K\ z) == \quad ;\ K\ z \rightarrow z$   
462  $z$

---

<sup>2</sup>Kleene (1952), p. 376, Theorem XXX. Four approaches to general-purpose computing were devised around the same time: Moses Schönfinkel and Haskell Curry's combinator SKI calculus (1920s); Alonzo Church's lambda calculus  $\lambda C$  (1936); Alan Turing's tape machine (1936); and Stephen Kleene's  $\mu$ -recursive functions (1936) building on Kurt Gödel's primitive recursive (1931) and general recursive functions (1934). These have been demonstrated to be equivalent to each other by means of the Church–Turing thesis: “Every effectively calculable function is a computable function.” Various auxiliary proofs establish the isomorphic nature of these systematic approaches; however, a footnote much larger than the length of this book is warranted to address the relationships amongst these four formulations. We simply take for granted their equivalence and move freely between representations as necessary. Furthermore, other formulations, such as the structured program theorem for control flow graphs (Böhm and Jacopini, 1966), exist and merit future exploration.

In such cases, combinators are applied from left to right by the first possible reduction. This means that they will bottom out in a particular expression, which may be the argument (as with  $I\ x == x$ ).

At this point, evaluation eagerness starts to matter: when we evaluate an expression, we need to decide which combinator to apply first or whether they apply at the “same time” (as with  $S\ K\ K$ ). The standard convention is to apply combinators from left to right, which means that we will evaluate the leftmost combinator first. This is known as leftmost reduction, and it ensures that we will eventually reach a normal form if one exists. However, there are other reduction strategies, such as rightmost reduction or parallel reduction, which can lead to different results in some cases. The choice of reduction strategy can affect the efficiency of evaluation and the order in which results are produced, but it does not affect the final result of the computation. One gentle exception to this is the  $Y$  combinator, which we will discuss in the next section; it is a fixed-point combinator that allows for recursion, and its behavior can be sensitive to the reduction strategy used.

Similarly,  $I$  can be expressed in terms of  $S$  and  $K$  alone:

$I\ x == S\ K\ K\ x$

It is of course frequently retained for economy, a harbinger of decisions to come.



BCKW makes these explicit as named combinators. SKI forces you to build duplication and composition from scratch each time using only substitution and constant. This is why BCKW is often preferred for studying the semantics of programming language features — each combinator names a specific capability, and you can ask “which features require W? which require C?” as a way of classifying programs.

### 2.1.3 $\Upsilon$ Fixed-Point

One thing that the SKI system lacks is a clean way to express recursion, or self-reference. This is because the combinators in SKI do not have any way to refer to themselves or to other combinators. To solve this problem, we can introduce a new combinator called  $\Upsilon$ , which is defined as follows:

---


$$\Upsilon f == f (\Upsilon f)$$


---

We call this a *fixed-point combinator* because it allows us to find a fixed point of a function, which is a value that is unchanged by the function. In other words, if we have a function  $f$  that takes a function as an argument and returns a function, then  $\Upsilon f$  will return a function that is a fixed point of  $f$ . This means that we can use  $\Upsilon$  to express recursion in the SKI system, which is essential for expressing many computable functions.

In SKI, self-reference requires  $\Upsilon$  because there’s no addressing — functions have no names, no locations, no way to refer

540 to themselves except by being passed to themselves explicitly.  
 541 That's what S I I (S I I) does: it hands a copy of itself to itself.

542 In Nock, self-reference is structural. The formula lives at  
 543 a known axis of the subject. Opcode o finds it. No Y needed  
 544 because the noun tree already encodes the fixed point — the  
 545 battery is always at axis 2, the sample at axis 6, and a recursive  
 546 call is just "retrieve axis 2 and evaluate it against a modified  
 547 subject." Y is the combinator-calculus workaround for the ab-  
 548 sence of addressing. Nock has addressing, so Y dissolves into  
 549 opcode o. For the book: This is worth a dedicated paragraph  
 550 in the combinators chapter. The argument is:

551 Y solves the problem of self-reference without names or ad-  
 552 dresses Nock has addresses — opcode o Therefore Nock doesn't  
 553 need Y; self-reference is structural not combinatorial Opcode  
 554 9 is syntactic sugar over opcode 2 over opcode o — peel the  
 555 layers and recursion is just tree addressing all the way down

556 Nock does not need a fixed-point combinator like Y because  
 557 it has a built-in mechanism for self-reference through its op-  
 558 code o, which retrieves the formula from a known axis of the  
 559 subject. This means that recursion in Nock is achieved through  
 560 structural self-reference rather than combinatorial self-reference,  
 561 and thus does not require a separate combinator to express it.  
 562 For the same self-referential reason, it is trivial to compose cer-  
 563 tain kinds of quines in Nock using opcode o, e.g.,

564 \*[[0 1] 0 1]

565 \*[[[0 2] 0 3] [[0 2] 0 3]]

566 Since Nock is a partial function, the result of subject–formula  
567 evaluation is a new noun, which can be fed back into the next  
568 evaluation as a new subject.

569 Any fixed-point combinator must somehow represent  $f$  ( $f$   
570 ( $f \dots$ )) to an infinite depth. The question is whether that  
571 infinite structure is eagerly materialized or lazily demanded—  
572 Nock’s referential structure facilitates the latter. (The real issue  
573 for  $Y$  in SKI terms is that it encodes infinite structure.)

574 Why, beyond the fact that it is a different basis, would we  
575 want to use BCK $\mathbb{W}$  instead of SKI? The answer is that BCK $\mathbb{W}$  has a  
576 more direct relationship to recursion, which is a fundamental  
577 aspect of computation. In particular, the  $Y$  combinator is the  
578 payoff that justifies caring about BCK $\mathbb{W}$  at all. The argument  
579 runs thusly:

- 580 1. SKI is Turing complete—but how do you get recursion?
- 581 2. You need  $Y$ —but  $Y$  in SKI is hideous
- 582 3. BCK $\mathbb{W}$  makes  $Y$  clean because  $\mathbb{W}$  is the duplication that re-  
583 cursion needs

584 Therefore BCK $\mathbb{W}$  is not just an alternative basis, it’s a more ex-  
585 planatory basis for why recursion works at all in a combinator  
586 system.

587 Then Nock opcode 9 is the punchline: it implements self-  
 588 referential arm invocation directly, which is essentially Y baked  
 589 into the hardware. You don't write Y in Nock because the call-  
 590 ing convention already assumes it. That's a genuine design  
 591 insight worth stating explicitly in the chapter.

592  $Y = S (K (S I I)) (S (S (K S) K) (K (S I I)))$

593 Two is the minimum for communication — a sender and a  
 594 receiver, a constant and a variable, K and S. Peirce's firstness  
 595 and secondness before thirdness can emerge. You could make  
 596 this case seriously without it becoming mysticism.

## 597 2.1.4 $\iota$ Universal

598 We are now prepared to confront Chris Barker's universal  $\iota$   
 599 combinator, which yields all the others.<sup>3</sup>  $\iota$  is truly weird and  
 600 has no direct analogue in human language or legible operators;  
 601 it represents a kind of ultimate compression, a single combina-  
 602 tor that can express all the others.  $\iota$  can be expressed as follows  
 603 in the SK system:

604  $\iota$   
 605  $x = x S K$

606 It's important to emphasize that this is an explanatory defini-  
 607 tion, not a constructive one. It tells us how  $\iota$  behaves, but we

---

<sup>3</sup>Barker (2001).



don't *need* S and K to define it, only as a description here. From this definition of  $\mathbf{I}$ , we can derive the other combinators, such as S and K (and *a fortiori* the rest):

$K = \mathbf{I} (\mathbf{I} (\mathbf{I} (\mathbf{I})))$

$S = \mathbf{I} (\mathbf{I} (K))$

The derivations are somewhat complex, but this gives you a flavor:

```

 $\mathbf{I}$ 
 $\mathbf{I} (\mathbf{I} (\mathbf{I} (\mathbf{I}))) x y = \mathbf{I} (\mathbf{I} (\mathbf{I} (\mathbf{I}))) S K x y$       -- apply outer  $\mathbf{I}$ 
                    =  $\mathbf{I} (\mathbf{I}) S K S K x y$       -- apply next  $\mathbf{I}$ 
                    =  $\mathbf{I} S K S K x y$           -- apply  $\mathbf{I} \mathbf{I} \rightarrow$ 
 $\mathbf{I} S K$ 
                    =  $S K K S K x y$           -- apply  $\mathbf{I}$  to S
                    =  $K x (K x) y$           -- S
reduction:  $SKKx = Kx (Kx)$ 
                    =  $x$                       -- K drops
second arg

```

$\mathbf{I}$  serves as a boundary condition, a proof that the search for minimality has a known terminus, but that terminus is not where we want to live.<sup>4</sup> Nor are these the only combinators that can be defined; Raymond Smullyan's *To Mock a Mocking-bird* (1985) is a treasure trove of combinator systems. For example, the Turing  $\Theta$  combinator can be used to express recursion without the need for  $\Upsilon$ . The number of combinator systems

<sup>4</sup>Barker from the beginning encourage the exploration of the  $\mathbf{I}$  combinator space.

632 is unbounded, and they can be constructed in various ways to  
633 suit different purposes.

634       That said, combinator calculi (and their cousin, the lambda  
635 calculus) are generally considered too minimalist for practical  
636 programming; they shine, however, in mathematical proofs.  
637 This means that we can use them to define a standard of be-  
638 havior for an algorithm, but implement the logic in a compliant  
639 form using whatever computational tools we like. This auster-  
640 ity motivates why Nock has twelve opcodes rather than merely  
641 one or two: practical expressiveness is the design criterion, not  
642 theoretical minimality. The spec is engineered to its purpose  
643 rather than being the smallest possible size. It is better to think  
644 of Nock not as a bare programming language, but as a standard  
645 of computation which can under many circumstances be used  
646 as the computation itself.<sup>5</sup>

647       Nock is also a changing specification, albeit slowly and  
648 within finite bounds. Nock embeds its own hyperstition by  
649 “kelvin versioning” or a cooling refinement: it is drawn inex-  
650 orably towards a particular future, and so it is not just a stan-  
651 dard of computation but a standard of the future of compu-  
652 tation. That is, at each revision, it becomes more and more  
653 difficult to change, until Nock finally freezes at revision oK.

---

<sup>5</sup>Consideration of when to do so is largely driven by performance bot-  
tlenecks.

## 2.2 Nock Opcodes as Combinator

At this point, we must engage Nock as a specification for computing. Nock defines a number of opcodes (“operation codes”) which span a range of common functional behaviors, including those we have seen in the basic combinators. Nock represents all data as a noun, which is either an atom (a natural number, which may be thought of a Gödel number or raw data as bytes) or a cell (an ordered pair of nouns, with a head and a tail). Nock opcodes operate with at least two arguments: a “subject” which defines the operating environment, more or less, and a “formula” or expression which in conjunction with the opcode describes the action to take. These may be arbitrarily composed to build programs.

The Nock specification is produced in full in Listing 2.1.

Listing 2.1: Nock 4K

Nock 4K

A noun is an atom or a cell. An atom is a natural number. A cell is an ordered pair of nouns.

Reduce by the first matching pattern; variables match any noun.

|         |           |
|---------|-----------|
| nock(a) | ★ a       |
| [a b c] | [a [b c]] |

## NOCKONOMICON

---

```

678 ?[a b]          0
679 ?a              1
680 +[a b]          +[a b]
681 +a              1 + a
682 =[a a]          0
683 =[a b]          1
684
685 /[1 a]           a
686 /[2 a b]         a
687 /[3 a b]         b
688 /[(a + a) b]     /[2 /[a b]]
689 /[(a + a + 1) b] /[3 /[a b]]
690 /a               /a
691
692 #[1 a b]          a
693 #[(a + a) b c]    #[a [b /[(a + a + 1) c]] c]
694 #[(a + a + 1) b c] #[a [/[(a + a) c] b] c]
695 #a                #a
696
697 *[a [b c] d]      [*[a b c] *[a d]]
698
699 *[a 0 b]          /[b a]
700 *[a 1 b]          b
701 *[a 2 b c]        *[*[a b] *[a c]]
702 *[a 3 b]          ?*[a b]
703 *[a 4 b]          +*[a b]
704 *[a 5 b c]        =[*[a b] *[a c]]
705
706 *[a 6 b c d]      *[a *[[c d] 0 *[[2 3] 0 *[a 4 4 b]]]]

```

```

707 * [a 7 b c]          * [* [a b] c]
708 * [a 8 b c]          * [[* [a b] a] c]
709 * [a 9 b c]          * [* [a c] 2 [0 1] 0 b]
710 * [a 10 [b c] d]     #[b * [a c] * [a d]]
711
712 * [a 11 [b c] d]      * [[* [a c] * [a d]] 0 3]
713 * [a 11 b c]         * [a c]
714
715 * a                   * a

```

The beginning of this specification is concerned with defining a noun, which is either an atom (a natural number) or a cell (an ordered pair of nouns). Several opcodes

Nouns are binary trees addressed using the natural numbers (in binary, the path down the tree is given by 0 for left and 1 for right) (see Figures 2.1 and 2.2).

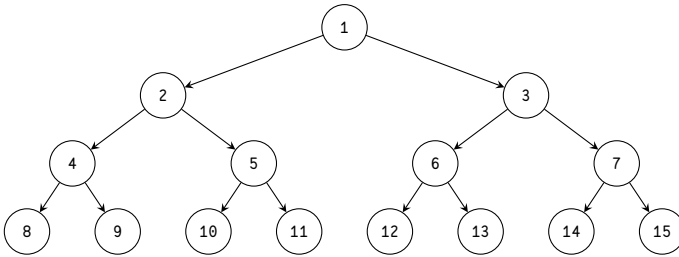


Figure 2.1: A binary tree with node addresses.

A Nock system has the following characteristics:

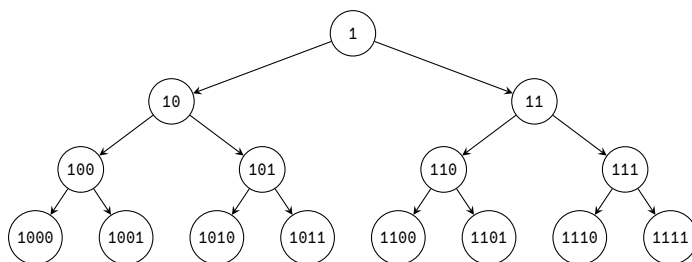


Figure 2.2: A binary tree with node addresses in binary indicating path.

- 723      1. Turing-complete: implements minimum requirements of  
724           $\mu$ -recursive functions. (This will be demonstrated be-  
725          low.)
- 726      2. Functional: uniquely maps [subject formula] to prod-  
727          uct. (This is a direct consequence of the definitions above.  
728          Nock evaluation is pure and deterministic.)
- 729      3. Subject-oriented: subject entirely determines scope for  
730          formula. (This is a direct consequence of the definitions  
731          above.)
- 732      4. Homoiconic: uses same representation for data and con-  
733          trol. (No distinction is made between code and data, in  
734          that both are nouns. Not all valid data is valid code, of  
735          course.)

5. Untyped: only knows about nouns. (While basic arithmetic is involved for cell addressing, no more algebraic machinery is imported.)
6. Solid-state: maintains no transient state in interpreter. (Each Nock evaluation is completely defined by its subject and formula. Noun ‘mutability’ is an apparent result of iterated evaluations of expressions on a subject.)

### 2.2.1 I Identity = Opcode 0

---

$I \ x \rightarrow x$

---

Nock’s binary tree addressing is fully general and yields the identity combinator I as a special case. (The cost, of course, is dragging in some machinery for basic arithmetic, as we will see—this is unnecessary in the base case but needed to describe and navigate nouns.)

Conceptually, we could write this in a hybrid form between combinators and Nock thus:

---

$I \ x \rightarrow \star[x \ 0 \ 1] ==$   
 $\quad \quad \quad / [1 \ x] ==$   
 $\quad \quad \quad x$

---

In a sense, I resolves immediately to  $[0 \ 1]$ , which is true but highlights a difference in terminology and argument structure. Nock conceives of itself as applying a formula to a subject,

762 which in combinator terms means the subject is one argument  
 763 and the formula(s) is/are another. However, the “position” of  
 764 each will not be consistent, so you cannot simply map  $x$  to  
 765 “formula”.

766 Nock’s homoiconicity, or the unification in representing  
 767 code and data, renders it distinct in some ways. The slot oper-  
 768 ator / fas is more general than combinators typically are, as it  
 769 assumes a structure to the input which is elided in combinatory  
 770 logic. This is a key difference between Nock and combinatory  
 771 logic: Nock is a language for manipulating structured data,  
 772 while combinatory logic is a language for manipulating func-  
 773 tions. The slot operator / fas is a way to access parts of struc-  
 774 tured data, which is not typically part of combinatory logic. In  
 775 our initial presentation, we are going to change the notation  
 776 of subject and formula around to better match the conventions  
 777 of the combinator definition for now.

## 778 2.2.2 K Constant = Opcode 1

---

779  
 780  $K \ x \ y \rightarrow x$   
 781

---

782 The constant combinator  $K$  is implemented in Nock as op-  
 783 code 1. It takes two arguments, a subject and a formula, and  
 784 returns the formula unchanged, ignoring the subject. This is  
 785 useful for building functions that always return a fixed value  
 786 regardless of the input.



---

```

787
788 K x y → *[y 1 x] ==
789           x

```

---

791 Note that we had to swap the order of  $x$  and  $y$  to match Nock's  
792 argument order. This is a common theme when translating  
793 between formal combinatory logic and Nock: the order of ar-  
794 guments may differ, so care must be taken to ensure that the  
795 correct values are being passed to the correct positions.

### 796 2.2.3 S Substitution = Opcode 2

---

```

797
798 S x y z → x z (y z)
799

```

---

800 The substitution combinator  $S$  is supplied in Nock as op-  
801 code 2, the evaluation operator. It takes three arguments, a  
802 subject and two formulas, and applies the first formula to the  
803 subject, then applies the second formula to the subject, and fi-  
804 nally applies the result of the first application to the result of  
805 the second application.

---

```

806
807 S x y z → *[z 2 x y] ==
808           *[*[z x] *[z y]]

```

---

### 810 2.2.4 The Axiomatic Operators

811 Because of Nock's cellular nature as a binary tree, we need the  
812 ability to access and manipulate nouns.

## NOCNONOMICON

---

813       The noun equality operator = tis is used to implement to  
814 opcode 5, which takes two arguments and returns 0 if they are  
815 equal (when evaluated against the subject) and 1 if they are  
816 not.

---

```
817  
818 :: TODO check  
819 = [x y] → *[z 5 x y] ==  
820            =[*[z x] *[z y]]
```

---

822       Nock supplies this as a primitive operation, but it can be  
823 built out of combinators as well if we supply alternative con-  
824 ventions for boolean values. In SKI, we represent true as K (i.e.,  
825 the first of two arguments) and false as SK (the second of two  
826 arguments).

### 827 Church Numerals in Nock

828       Classical combinatory logic does not build in the natural num-  
829 bers, but derives them as Church numerals. Church numer-  
830 als are a clever way to represent natural numbers as func-  
831 tions. They are more naturally expressed in the lambda cal-  
832 culus, but can be converted from  $\lambda C$  to our preferred SKI basis.  
833       The Church numeral for zero is  $\lambda f . \lambda x . x$ , which means that it  
834 takes a function  $f$  and a value  $x$ , and returns  $x$  unchanged. The  
835 Church numeral for one is  $\lambda f . \lambda x . f \ x$ , which means that it  
836 takes a function  $f$  and a value  $x$ , and applies  $f$  to  $x$  once. The  
837 Church numeral for two is  $\lambda f . \lambda x . f \ (f \ x)$ , which means that

838 it takes a function  $f$  and a value  $x$ , and applies  $f$  to  $x$  twice, and  
839 so forth.

840 To build Church numerals in SKI, we can use the following  
841 translations from  $\lambda C$  (presented without derivation from  $\lambda C$  to  
842 SKI):

843  $0 = K I$   
844  $1 = I$   
845  $2 = S (S (K S) K) I$   
846  $3 = S (S (K S) K) (S (S (K S) K) I)$

847 To cast Church numerals into Nock terms, let's apply our ex-  
848 pressions for  $I$ ,  $K$ , and  $S$ . (We face our next challenge in trans-  
849 lating from SKI to Nock, which is that SKI evaluates from left  
850 to right, while Nock evaluates from right ("innermost") to left  
851 ("outermost").) We also need to distinguish a Church numeral  
852 from the natural number to which it corresponds.

853  $0 = K I \Rightarrow * [y K I] == * [y 1 [0 1]] == [0 1]$   
854  $1 = * [y I] == * [y 0 1] == /[1 y]$   
855  $2 = * [z S I] == * [z 2 [0 1] [0 1]] == * [* [z [0 1]]$   
856  $\quad * [z [0 1]]]$   
857  $3 = * [z S (S (K S) K) I] == * [z 2 [2 [1 [0 3] [0 2]]$   
858  $\quad [0 1]]]$   
859  $\quad == * [* [z [2 [1 [0 3] [0 2]]]] * [z [0 1]]]$   
860  $\quad == * [* [* [z 2 [1 [0 3] [0 2]]] * [z 1 [0 1]]] * [z$   
861  $\quad [0 1$   
862  $\quad ]]]$   
863  $\quad == * [* [* [* [z 2 [0 3]] * [z 2 [0 2]]] * [z 1 [0 1]]]$   
864  $\quad * [z [0 1]]]$

## NOCKONOMICON

---

```
865      == *[*[*[*[*[z 2 [0 3]] *z 2 [0 2]]] *[*[z 1 [0
866          1]]]] *z [0 1]]]
867      == *[*[*[*[*[z 2 [0 3]] *z 2 [0 2]]] *[*[z 1 [0
868          1]]]] *z [0 1]]
```

869     A general-purpose increment operator can be built in SKI  
870 as well. Let's call this increment operator A (classically written  
871 Succ) and derive it as follows:

---

```
872 :: S(S(KS)K)(S(KK)I)
873
```

---

875 Very awkward! Fortunately we don't need to continue down  
876 this road. We don't actually need Church numerals for Nock  
877 because Nock has built-in arithmetic, but they are a useful ex-  
878 ercise for understanding how to build more complex combina-  
879 tors.

### 880 Increment, Opcode 4

881 Given that the natural numbers are built into Nock via both  
882 atoms and the tree addressing scheme, we can change an atom  
883 to its successor by applying the increment operator + (opcode  
884 4). This takes a single argument, an atom, and returns the next  
885 natural number.<sup>6</sup>

---

<sup>6</sup>This will always succeed for an atom, since every natural number has a successor. If the argument is a cell, it will fail, as +[a b] spins.

## Structure Checks, Opcodes 3 and 5

Two salient structure checks are provided in Nock: a cell check, which results in TRUE/0 if the argument is a cell and FALSE/1 otherwise (only an atom); and an equality check, which results in TRUE/0 if the two arguments are equal and FALSE/1 otherwise. These are implemented as opcodes 3 and 5, respectively.

There are not equivalent combinators in the SKI calculus for these checks since because SKI has no notion of structure for its arguments. SKI operates on functions as black boxes and cannot inspect whether an argument is a pair or an atom.

(This pair of opcodes implies the possible desirability of a general-purpose structural check in Nock, which would match the overall binary tree layout of two nouns but not check the actual values of the atom leafs. Thus [0 1 2] would match [0 1 2] and [0 1 3], but not [[0 1] 2 3] or [0 1].)

### 2.2.5 Pythagoras and Peirce in Nock

*Die ganzen Zahlen hat der liebe Gott gemacht, alles andere ist Menschenwerk.*

The dear Lord created the natural numbers; all else is the work of Man.

(Leopold Kronecker, 1886 lecture to Berlin Naturalists' Assembly)

The first is that whose being is simply in itself, not

909       referring to anything nor lying behind anything.  
 910       The second is that which is what it is by force of  
 911       something to which it is second. The third is that  
 912       which is what it is owing to things between which  
 913       it mediates and which it brings into relation to  
 914       each other.  
 915       (Charles Sanders Peirce, *Collected Papers* 2.356)

916       To take an esoteric aside on the nature of Nock as a compu-  
 917       tational platform, we may consider the Pythagorean relation-  
 918       ship of the axioms and the opcodes.<sup>7</sup> Kronecker’s quip was a  
 919       bit of hyperbole leveled at Georg Cantor’s number mysticism,  
 920       but it nevertheless highlights the spartan utility of atoms and  
 921       addressing in Nock.

922       The natural numbers are conventionally taken as a con-  
 923       structive ordered set of 0 and its successors along with addi-  
 924       tion, multiplication, and division (either floor division or di-  
 925       vision with remainder). (There is no additive inverse or sub-  
 926       traction in the natural numbers, since subtraction may result  
 927       in a value that is not a natural number.) Nock dispenses with  
 928       multiplication and division, relying strictly on addition in its  
 929       axiomatic definitions. Nock 0 as an addressing system already  
 930       in fact carries within it the seeds of the axiomatic operators ?

---

<sup>7</sup>In a general way, ~barpub - tarber and his Conk formulation as the field of possible Nocks (developed 2013–2017, republished 2025) parallel this analysis at the strict level of the opcodes.

931 and +, because the addressed binary tree must know if a leaf is  
932 a cell or atom and be able to calculate its head and tail via use  
933 of the successor function +.

934 Furthermore, there is a sense in which computing is deeply  
935 semiotic. Semiotics is the philosophy of symbols and how they  
936 interrelate. (At the outside, you end up with language, culture,  
937 and thought itself as mutually recursive tangle of sign refer-  
938 ences, but we don't need to go that far here.<sup>8</sup>) Here we simply  
939 note that any noun always stands in relation to (at least) two  
940 possibilities: being a subject or being a formula. Of course, not  
941 every noun can be evaluated as a valid formula, but any noun  
942 can be utilized as a subject for appropriate formulas. Thus a  
943 noun is a member of an evaluation system: its purpose is to  
944 participate in a computation.<sup>9</sup>

945 Peirce's triadic semiotics gives us a way to conceive of nouns  
946 as embedded symbols. No noun is an island: it may be consid-  
947 ered by itself, but it always bears a relationship to other nouns,  
948 even possible nouns. So it has both an identity and a contrast.  
949 Harkening back to its role as a subject or possibly as a for-  
950 mula, a noun naturally carries Peirce's thirdness: it must stand

---

<sup>8</sup>See the works of Saussure, Peirce, Eco, and Derrida, if you are interested in a great rabbit hole.

<sup>9</sup>Another stance of nouns to consider is with respect to the field of possible binary trees, that is, the field of possible nouns. We briefly note that the first possible formula is the crash or void, [0 0], and the second possible formula is the identity, [0 1].

951 in relation to other nouns.

952       Numbers as first explicated by Pythagoras remain magical  
953 when taken sufficiently seriously. Nock catches one more facet  
954 of their light.

### 955 **2.3 Idioms & Design Patterns**

956 Several of the combinators we examined above are not present  
957 in Nock *simpliciter*, but we can build them out of the other  
958 pieces. There are also common idioms which are baked in as  
959 “compound opcodes” (such as conditional branching, opcode  
960 6).<sup>10</sup> Let’s look at a few common combinators and then extend  
961 Nock with its standard opcodes.

962       We can approach the construction of equivalent combina-  
963 tor expressions in Nock in two ways. The first is to start with  
964 the combinator definition in terms of SKI and then substitute  
965 our Nock expressions for S, K, and I. The second is to construct  
966 the simplest Nock expression, which utilizes the additional af-  
967 fordances of Nock (primarily slot addressing with opcode o) to  
968 achieve the same effect.

#### 969 **2.3.1 C Flip Operator**

---

<sup>10</sup>This first set of combinators derives from the original Schoenfinkel-Curry work (Curry, 1930).



---

```

970 C x y z == x z y
971
972

```

---

973     The flip operator `C` is a combinator that takes a pair of argu-  
974     ments and reverses their order. It is useful for building func-  
975     tions that take two arguments and apply them in a different  
976     order than they are given. In SKI terms, we can express this as  
977     follows:

---

```

978 C = S (S (K (S (K S) K)) S) (K K)
979
980

```

---

981     While we can formulate the flip operator `C` in terms of the  
982     Nock translation of `S` and `K`, it is far more practical to use op-  
983     code `o`. This is almost free due to the binary tree nature of cells.  
984     In Nock, we express this as follows:

---

```

985 C x y z == x z y →
986
987     [*[*[y z] [0 3] x] [*[y z] [0 2] x]]
988

```

---

989     `C` simply rearranges the order of two nouns, which can be use-  
990     ful in managing conditional branches or modifying function  
991     argument order.<sup>11</sup> (For example, there is an operation called  
992     *currying* we'll see later which takes a function of two argu-  
993     ments and returns a function of one argument that applies the  
994     first argument to the second. If you wanted to bind the second  
995     argument, however, you could use something like `C` to flip the  
996     order of the arguments when you curry.)

---

<sup>11</sup>You can see here how using opcode `o` substantially simplifies the Nock  
equivalent expression versus the SKI definition.

## 997 2.3.2 B Composition Operator

998  
 999 B x y z == x (y z)  
 1000

---

1001 The composition operator B is a combinator that takes three  
 1002 arguments: a function and two values. It applies the second  
 1003 value to the third value, then applies the function to the result.  
 1004 In SKI terms, we can express this as follows:

1005  
 1006 B = S (K S) K  
 1007

---

1008 The more straightforward Nock expression for B in terms  
 1009 of S and K would be:

1010  
 1011 B x y z →  
 1012       \*[z 7 x y] ==  
 1013       \*[\*[z y] x]

---

1015 This is very similar to opcode 7, modulo rearranging the argu-  
 1016 ment order.

1017 B is useful for building functions that apply a function to  
 1018 the result of another function, allowing for more complex com-  
 1019 positions of functions.

## 1020 2.3.3 Join Operator

1021  
 1022 Join x y == x y y  
 1023

---

The join operator  $\mathbb{J}$  is a combinator that takes two arguments: a function and a value. It applies the function to the value twice; that is, it duplicates its second argument and applies it twice to the first. In SKI terms, we can express this as follows:

---


$$\mathbb{J} = S S (S K)$$


---

The more straightforward Nock expression for  $\mathbb{J}$  in terms of S and K would be:

---


$$\begin{aligned} \mathbb{J} \ x \ y \rightarrow \\ & \quad *[*[y \ x] \ y] \\ & \quad *[y \ * [y \ x]] \end{aligned}$$


---

$[.*(42 \ [4 \ 0 \ 1]) \ 42]$

This is very similar to opcode 8, modulo rearranging the argument order. Opcode 8 also has a built-in continuation to apply the result to a third argument.

### 2.3.4 $\mathbb{U}$ Duplication Operator

<sup>12</sup>

---

<sup>12</sup>The second set of combinators follows work done by Smullyan (1985) in his marvelous *To Mock a Mockingbird*.

## 1045 **2.4 Nock as Generally Computable**

1046 While it is beyond our scope to prove the equivalence of the  
 1047 four formulations of general computability, we can briefly demon-  
 1048 strate that Nock does satisfy the requirements of each.

1049 SKI and  $\lambda$ C completeness have been established above. Tur-  
 1050 ing machine equivalence is established by the fact that we can  
 1051 build a universal Turing machine in Nock. Per <sup>\*</sup>(\*)Hopcroft1979,  
 1052 p. 148, a universal Turing machine has the following compo-  
 1053 nents:

- 1054 • A finite, non-empty set of tape alphabet symbols, includ-  
 1055 ing a blank symbol.
- 1056 • A finite, non-empty set of states, including a start state  
 1057 and one or more halting states.
- 1058 • A transition function that, given the current state and  
 1059 the symbol being read, specifies the next state, the sym-  
 1060 bol to write, and the direction to move (left or right).

1061 Nock as operator over nouns satisfies these requirements with  
 1062 the natural numbers (including crash [0 0] as blank) and cells  
 1063 to constitute nouns, while the transition function is represented  
 1064 by the evaluation rules of Nock itself.

1065 Finally, Nock is generally recursive (or  $\mu$ -recursive) be-  
1066 cause it has the necessary building blocks:<sup>13</sup>

- 1067 • Constant function including zero (opcode 1)
- 1068 • Increment or successor function (opcode 4)
- 1069 • Parameter selection or variable access (opcodes 0 and 9  
1070 along with opcodes 7 and 8)
- 1071 • Function composition or program concatenation (opcodes  
1072 7 and 8)
- 1073 • Recursion or looping (opcode 0 with, e.g., opcodes 3, 5,  
1074 and 6)

1075 Thus Nock can express all  $\mu$ -recursive functions.

## 1076 2.5 A Stateful Nock Kernel

1077 Nock is a crash-only pure language, which means at the base  
1078 level it either fail (and crash) or succeed (and yield a new noun):  
1079 there are no side effects, no exceptions, and no state. Like a  
1080 castaway on a desert island, we must build everything we need  
1081 from the raw materials at hand.

---

<sup>13</sup>Boolos, Burgess, and Jeffrey (2002), “6.2 Minimization”, pp. 70–71.

1082        Thus far, we have said nothing practical about combina-  
1083 tors: they live in an authochthonous sphere, hermetically sealed  
1084 from the environment. This is what is meant by “purity” in the  
1085 functional programming sense: the result of a function is en-  
1086 tirely determined by its explicit input arguments, without any  
1087 side effects or reliance on external state. This is a powerful  
1088 property that allows for easier reasoning about code and en-  
1089 ables certain optimizations, but it also means that we need to  
1090 find ways to interact with the outside world and maintain state  
1091 across computations.

1092        Nock provides us with a single escape hatch, a set of flags  
1093 to the outside world which trigger impure side effects: opcode  
1094 11. Combined with a couple of design patterns, we can thereby  
1095 escape the autochthon and construct a practical Nock kernel  
1096 that can maintain state and interact with the environment.

- 1097        1. Opcode 11 lets us send a hint to the runtime, which can  
1098        be used to trigger side effects. The Nock expression for-  
1099        mally doesn’t know anything about these, but we can  
1100        use them to print results, request input, and so forth.
- 1101        2. The runtime can maintain state outside of Nock and sup-  
1102        ply it as a subject to subsequent Nock expressions. We  
1103        will do a lot more with this later, but we want to high-  
1104        light it as a key affordance now.
- 1105        3. The virtualization pattern means that we can run Nock

1106 in Nock.

1107 Opcode 11 can encode anything we want: the result of a  
 1108 hint is used by the runtime, and so anything the runtime is  
 1109 willing to work with can be produced.

1110 The magic formula at the beating heart of a practical Nock  
 1111 operating system is simply:

---

```
1112 *([state -.events] [9 2 10 [6 0 3] 0 2])
```

---

1115 That is, apply the Nock formula against a subject pair consist-  
 1116 ing of the current state and the head of the list of events to  
 1117 process.

1118 <https://web.archive.org/web/20070716223000/https://homepages.nyu.edu/cb1>  
 1119 <https://komiamiko.me/math/ordinals/2020/06/21/ski-numerals.html>  
 1120 <https://www.angelfire.com/tx4/cus/combinator/birds.html> <https://www.sciencedirect.com>

1121





---

## Chapter 3 The Virtues

Metacircularity in most languages is a curiosity and corner case. Programmers are routinely warned not to use it, and for good reason. Practical metacircularity is a particular strength of Nock, or any functional assembly language. Dynamic loading and/or virtualization works, at least logically, as in an OS.

... A metacircular functional dissemination protocol can serve as a complete, general-purpose system software environment—there is no problem it cannot solve, in theory or in practice. Such a protocol can be described as an “operating function” or OF—the functional analog of an imperative operating system. (~s0rreg-namtyv, 2010)

Nock is a crash-only language: the first coherent formula is the pair  $[0\ 0]$ , the crash. There is no error handling in raw

1139 Nock, no exceptions. On the one hand, this is a strength: a  
1140 Nock noun produces no formal side effects, and may be tried  
1141 again with a different subject or formula if it fails. On the  
1142 other, naïve Nock has no way to recover from failure. The trick  
1143 to doing this correctly is to operate virtually, to produce a Nock  
1144 interpreter that runs on top of Nock itself, and to use this in-  
1145 terpreter to run Nock programs. This allows us to catch errors  
1146 and respond to them in a controlled way, while still maintain-  
1147 ing the purity and simplicity of Nock as a language.<sup>1</sup>

1148       What could we want of a virtualized Nock interpreter? The  
1149 most important factor is error handling, but we can build more  
1150 complex macros, even new opcodes, into any interpreter as  
1151 well.

1152       While virtualization necessarily incurs some overhead rel-  
1153 ative to more direct execution, it gives the Nock programmer  
1154 a way to bail (crash), trace (log with debugging information),  
1155 and bind (limit in some sense<sup>2</sup>) a computation. In fact, stack  
1156 traces are computed through virtualization, and so are guar-  
1157 anteed to be as correct as their implementation.

1158       A Nock virtualization environment needs to distinguish at  
1159 least three kinds of results:

- 1160       1. A successful result of computation, along with a valid

---

<sup>1</sup>This chapter draws on ~lacnes (2025) with the blessing of author and publisher. Incidental quotations are made without attribution for readability.

<sup>2</sup>Like stack depth.

1161       Nock noun.

1162       2. A block, which indicates that the computation is not yet  
1163       complete and should be continued.

1164       3. A halt or deterministic error, which indicates that the  
1165       computation has failed and should not be continued.

1166       Traditionally, opcode 12 has been used to permit the hyper-  
1167       Turing “scry” operator, which allows you to look up a bound  
1168       persistent value in a referentially transparent way. This is es-  
1169       pecially useful when a necessary value is not available in the  
1170       current subject or needs to be supplied by the runtime (such  
1171       as a system date or entropy).

1172       We can also implement the combinators as extended Nock  
1173       opcodes, aiding us in building more complex expressions from  
1174       our higher-level language compiler when we get there.

1175       13. B composition operator

1176       14. C flip operator

1177       15.  join operator

1178       16. U duplication operator

1179       17. XXX



---

## Chapter 4 Interpreter Runtime

### Abstract

Now that we have established a firm foundation for idiomatic programming in Nock, we can build an interpreter using a higher-level language to execute Nock formulas. At this point, we will build the simplest possible Nock interpreter, a tree-walking interpreter. This requires a cursor and some basic memory management but not much more. We will implement some minimalist static hints to permit side effects like output.



---

## Chapter 5 Moon: A Higher-Level Language

### Abstract

While somewhat more economical than a minimal combinator language, Nock is still quite low-level and cumbersome as we have seen. In this chapter, we will introduce a higher-level language called *Moon* which compiles to a Nock noun.<sup>1</sup>

### 5.1 Relationship of Moon to Nock

You can simply understand our goal as to produce executable Nock nouns using a more legible and flexible syntax than pure Nock affords. But it's worth casting Moon in the context of Nock's evaluation model.

---

<sup>1</sup>The name *Moon* is a double entendre: it is a micro-*Hoon*, the primary systems language of Urbit and NockApp applications, and it references Kleene's  $\mu$ -recursive functions.

1200       The Moon compiler is a Nock noun which (as subject) oper-  
1201       ates on a text string to produce a new Nock noun. This second  
1202       noun is itself a formula against which well-structured formulas  
1203       may be evaluated. Formally,

---

```
1204       Moon(source) == Nock(source moon-formula)
```

---

1207       that is, the evaluation of the Moon compiler as a subject against  
1208       a source string produces a Nock noun which is the compiled  
1209       output of the Moon compiler. We can think of this as a two-  
1210       step process: first, we compile the source code to a Nock noun,  
1211       and then we evaluate other Nock nouns against that noun.

1212       where `moon-source` is a text string containing the Moon source  
1213       code, and `moon-formula` is a Nock noun which is the compiled  
1214       output of the Moon compiler. The Moon compiler is itself a  
1215       Nock noun, and so we can think of it as a self-compiling com-  
1216       piler.<sup>2</sup>

1217       We will have to construct Moon in two stages: a launch  
1218       stage (“New Moon”) which can only compile simple expres-  
1219       sions, a complex construction exemplifying all that we’ve seen;  
1220       and a payload stage (“Crescent Moon”) which can compile it-  
1221       self and thus becomes a Nockhound’s flywheel (“Full Moon”  
1222       being the completed product). We have two possible ways to  
1223       construct New Moon as the launch stage: either we can write

---

<sup>2</sup>We here follow the exposition of **Yarvin2010c** (**Yarvin2010c**) in his description the a high-level language compiler to Nock for the first time.



the Moon compiler directly in bespoke Nock by hand, or we can compose it in the same language as our interpreter, Rust.

## 5.2 ☒ New Moon

### 5.2.1 Design Criteria

New Moon is complete when it can compile some set of simple expressions to Nock. It does not yet, however, need to compile itself, which we defer until Crescent Moon.

Since Nock possesses its own set of idioms (as we've worked out in the preceding chapters), we know what the Nock products need to look like. A higher-level language can take into account the structure of its target ISA in its syntax or not, but we will find it easier to write Moon if we do. (Hoon takes this road, being essentially a macro language and type system over Nock.)

For instance, in Nock we would apply a function by loading the reference via opcode 9 and then replacing its sample with the new arguments. In Moon, we need a way to express this intent in an unambiguous way; let's say that we take a page out of Lisp's book and use parentheses to indicate function application. Thus, we could write a hypothetical Moon expression like this:

---

```
(+ 1 41)
```

---

1248 That's reasonable if Lisp-flavored; indeed, we will find that  
1249 Moon has Lisp-like characteristics. From a parsing standpoint,  
1250 this needs to result in an abstract syntax tree (AST) which de-  
1251 scribes the operation and its arguments, thus detecting the  
1252 head of the expression as its function and the tail to the ending  
1253 parenthesis as arguments. We'll use a syntax in which text is  
1254 marked by a % cen prefix, resulting in the hypothetical AST:

1255  
1256 [%add 1 41]  
1257

---

1258 and ultimately in the Nock noun (modulo tree addresses):

1259  
1260 [8 [9 cor 0 arm] 9 2 10 [6 1 1 41] 0 2]  
1261

---

1262 (Read it carefully—we pin the core locally with an opcode 8,  
1263 then we modify the local copy with opcode 10, replacing the  
1264 sample with the new arguments.)

1265     You can follow a thread of logic from the initial expression  
1266 in Moon to the final Nock noun, mediated by the AST repre-  
1267 sentation. New Moon thus has two phases: a parser and a  
1268 compiler. We can build them separately, but first let's look at  
1269 more examples and their corresponding AST representations.

### 1270 5.2.2 New Moon Examples

1271  
1272 :: This is a comment. It doesn't affect the AST or  
1273 code.  
1274

```
1275 :: This is a simple expression.  
1276 (+ 1 41)  
1277  
1278 :: This is a more complex expression.  
1279 (- 54 (+ 3 9))  
1280  
1281 :: This is a variable declaration.  
1282 (= a 42)  
1283  
1284 :: This is a loop with a check.  
1285 for (= a 0)  
1286 =a (- )  
1287
```

---

### 1288 5.2.3 Parsing New Moon

1289 We will soon run out of symbols: the punctuation in ASCII is  
1290 limited. We could choose to switch to words throughout or to  
1291 permit paired punctuation; we will do both eventually but for  
1292 now we'll let it ride.<sup>3</sup>

---

<sup>3</sup>Another alternative is to support Unicode characters, as Julia does, but parsing UTF-8 text is more complex than Moon will support out of the box.

#### 1293 **5.2.4 Compiling New Moon**

### 1294 **5.3 ☒ Crescent Moon**

1295 The objective of Crescent Moon is to bootstrap, or compile it-  
1296 self. Once we have this capability, expanding Moon to a full  
1297 language becomes simply a matter of adding more syntax and  
1298 semantics to the compiler. This means that Crescent Moon  
1299 must be able to parse and compile text, with any required types  
1300 being supplied as well.

1301 At this point the question of the target ISA becomes acute:  
1302 should Moon compile to strict Nock or to our extended Mock?  
1303 That is, if our compiler utilizes either extended opcodes 6–9  
1304 or Mock opcodes 12–XX, how will it relate to the runtime sys-  
1305 tem? At the end of the day, Nock is Nock (since we can always  
1306 reduce), but we must make an aesthetic design choice which  
1307 may have deep implications for Moon.

1308 Let's consider what Hoon does. Hoon actually targets Mock  
1309 (with opcode 12 only) and does some sensible reductions and  
1310 compile-time checks. For instance, Hoon compilation of op-  
1311 code 6 can eliminate branches if the conditional reduces to an  
1312 opcode 1 (see `+cond`). The runtime compiler and system can  
1313 make further optimizations: opcode 11 `%fast` hints for jets are  
1314 only checked for opcode 9 expressions, not for opcode 2. Nock  
1315 7 is compiled away entirely.

1316 At this point, we are not interested in optimizations, but

there's no reason to forswear their future possibility. We will therefore make the following design decision: Moon targets canonical Nock without Mock opcodes (12 and higher). This will make Moon somewhat less efficient than Hoon as a systems language, but it will remain more legible.

## 5.4 ☒ Full Moon

From Crescent Moon, building Full Moon becomes a matter of expanding the language syntax, elaborating more parsers and other tools, and producing libraries of useful functions.

(at this point, back out from parentheses to centis for function application)

vase mode and type system redux

What other languages compile to Nock? The earliest higher-level language was Watt, the ancestor of Hoon. Hoon Jock currying etc.



---

## Chapter 6 A Virtual Machine Runtime

**The Kraken** Below the thunders of the upper deep,  
Far, far beneath in the abysmal sea, His ancient,  
dreamless, uninvaded sleep The Kraken sleepeth:  
faintest sunlights flee About his shadowy sides;  
above him swell Huge sponges of millennial growth  
and height; And far away into the sickly light, From  
many a wondrous grot and secret cell Unnumbered  
and enormous polypi Winnow with giant arms the  
slumbering green. There hath he lain for ages,  
and will lie Battening upon huge sea worms in his  
sleep, Until the latter fire shall heat the deep; Then  
once by man and angels to be seen, In roaring he  
shall rise and on the surface die. (Alfred, Lord Ten-  
nyson)

To date, we have run Nock either by hand or using a simple  
interpreter. While this suffices for relatively small programs,

N. E. Davis ~lagrev-nocfep. “A Virtual Machine Runtime”. *Nockonomicon*  
(2025): 50–x.

1349 our objective of building a Nock computer requires an “oper-  
1350 ating function”, or persistent event loop. This chapter intro-  
1351 duces the considerations of running Nock continuously and  
1352 reactively as well as how to maintain state and respond to op-  
1353 code 11 hints.

## 1354 **6.1 The Event Loop**

1355 A single Nock expression as a subject–formula pair accepts  
1356 two nouns and yields a new noun. We can think of this new  
1357 noun as the total state of a program or an operating system,  
1358 feeding it into the next expression as the subject for a new for-  
1359 mula to evaluate against. This preserves the determinism and  
1360 legibility of Nock while allowing for complex state transitions.

### 1361 **6.1.1 A Simple State Machine**

1362 A state machine is a mathematical abstraction for a system that  
1363 has a definite state at any particular time, and which changes  
1364 in response to inputs.

#### 1365 **Example: Stoplight**

1366 Think of a stoplight, which has a state of red, yellow, or green,  
1367 and which changes its state in response to an external timer or  
1368 a button press.



1369 **Example: Revolver**

1370 Or again, of a six-shot revolver, which if we include chamber  
1371 order has 64 finite state permutations with thirteen possible  
1372 inputs—to “load” or “clear” at each chamber and to “fire”—and  
1373 two possible outputs—a “bang” or a “click” accompanied by a  
1374 new state.

1375 Aggregate or macro actions are also available—a “moon  
1376 clip” corresponds to loading all six chambers at once, and a  
1377 “dump cylinder” corresponds to clearing all six chambers at  
1378 once. The revolver is a state machine with a finite number of  
1379 states, inputs, and outputs.

1380 1

1381 We will call a state machine which adheres to the defini-  
1382 tion of its lifecycle function a *kernel*. Thus a kernel is a state  
1383 machine of a certain shape which accepts nouns of a certain  
1384 shape as input and produces nouns of a certain shape as out-  
1385 put. The kernel is the event loop and some wrapper of library  
1386 functionality to bound and define the complete behavior of the  
1387 state machine. A kernel may be batch or continuous depend-  
1388 ing on whether its lifecycle function is reentrant.

---

<sup>1</sup>+`poke` and +`peek` derive from a very old convention in BASIC, wherein `POKE` and `PEEK` commands allowed programmers to write to and read from arbitrary memory addresses.

1389 **6.1.2 An Evolved State Machine**

1390 While the two-armed model provides a straightforward way to  
1391 manage state transitions, Nockhounds have found in practice  
1392 that it is more convenient to supply four arms at the following  
1393 axes:

1394     +4: `+load` is used in kernel upgrades, allowing the event loop  
1395         to update itself live.

1396     +10: `+wish` accepts a core and parses it against a standard li-  
1397         brary for the kernel.

1398     +22: `+peek` grants read-only access to the system's state; this  
1399         is often called a "scry".

1400     +23: `+poke` accepts events and processes them, resulting in a  
1401         new state noun.

## Bibliography

- Barker, Chris (2001). *Iota and Jot: the simplest languages?* URL:  
<https://web.archive.org/web/20160823182917/http://semarch.linguistics.fas.nyu.edu/barker/Iota/>.
- Böhm, Corrado and Giuseppe Jacopini (1966). “Flow Diagrams, Turing Machines and Languages with Only Two Formation Rules.” In: *Communications of the ACM* 9.5, pp. 366–371. ISSN: 0001-0782. DOI: 10.1145/355592.365646.
- Boolos, George, John Burgess, and Richard Jeffrey (2002). *Computability and Logic*. 4th. Cambridge: Cambridge University Press. ISBN: 9780521007580.
- Curry, Haskell (1930). “Grundlagen der kombinatorischen Logik.” In: *American Journal of Mathematics* 52.3, pp. 509–536, 789–834. ISSN: 0002-9327. DOI: 10.2307/2370619.
- Hofstadter, Douglas R. (1980). *Gödel, Escher, Bach: An Eternal Golden Braid*. New York: Basic Books.

- 1420 Hopcroft, John E. and Jeffrey D. Ullman (1979). *Introduction to*  
 1421 *Automata Theory, Languages, and Computation*. Reading,  
 1422 MA: Addison-Wesley.
- 1423 Kleene, Stephen Cole (1952). *Introduction to Metamathematics*.  
 1424 Amsterdam: North-Holland.
- 1425 ~lagnes (2025). “Metacircular Virtualization and Practical  
 1426 Nock Interpretation.” In: *Urbis Systems Technical Journal*  
 1427 3.1, pp. 71–97.
- 1428 Penrose, Roger (2004). *The Road to Reality: A Complete Guide*  
 1429 *to the Laws of the Universe*. New York: Vintage.
- 1430 Perlis, Alan J. (1982). “Epigrams on Programming.” In: *ACM*  
 1431 *SIGPLAN Notices* 17.9, pp. 7–13. ISSN: 0362-1340. DOI:  
 1432 10.1145/947955.1083808.
- 1433 Smullyan, Raymond (1985). *To Mock a Mockingbird and Other*  
 1434 *Logic Puzzles: Including an Amazing Adventure in*  
 1435 *Combinatory Logic*. New York: Knopf.
- 1436 Wolfram, Stephen (2002). *A New Kind of Science*. Champaign,  
 1437 IL: Wolfram Media.
- 1438 — (2021). *The Concept of the Ruliad*. URL:  
 1439 [https://writings.stephenwolfram.com/2021/11/the-](https://writings.stephenwolfram.com/2021/11/the-concept-of-the-ruliad/)  
 1440 [concept-of-the-ruliad/](https://writings.stephenwolfram.com/2021/11/the-concept-of-the-ruliad/).